

- Relatório de Análise de Algoritmos de Ordenação
 - 1. Introdução
 - 2. Tabela de Resultados
 - 3. Análise dos Resultados e Discussão
 - 3.1. Cenário Aleatório
 - 3.2. Cenário Crescente
 - 3.3. Cenário Decrescente
 - 4. Conclusão

Relatório de Análise de Algoritmos de Ordenação

Disciplina: Resolução de Problemas Estruturados em Computação **Estudantes:** Vitor, Yas e Bia **Professora:** Marina de Lara

1. Introdução

Este relatório apresenta a análise de eficiência dos algoritmos de ordenação **Bubble Sort**, **Insertion Sort** e **Quick Sort**. O objetivo principal é avaliar o comportamento de cada algoritmo ao processar conjuntos de dados de diferentes tamanhos (100, 1.000 e 10.000 elementos) sob três condições de ordenação distintas: dados aleatórios, dados previamente ordenados de forma crescente e dados ordenados de forma decrescente[cite: 14, 15, 16]. Os tempos de execução foram medidos via `System.nanoTime()` e convertidos para milissegundos (*ms*) para fins de análise estruturada.

2. Tabela de Resultados

A tabela abaixo consolida os tempos de execução coletados durante a execução do programa para cada cenário testado:

Arquivo de Dados	Tamanho (N)	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
aleatorio_100.csv	100	0,147	0,045	0,023
aleatorio_1000.csv	1.000	3,874	1,467	0,207
aleatorio_10000.csv	10.000	61,486	18,231	0,772
crescente_100.csv	100	0,003	0,008	0,011
crescente_1000.csv	1.000	0,181	0,025	0,288
crescente_10000.csv	10.000	17,084	0,056	26,725
decrescente_100.csv	100	0,009	0,004	0,008
decrescente_1000.csv	1.000	0,908	0,105	0,216
decrescente_10000.csv	10.000	92,649	9,292	20,500

3. Análise dos Resultados e Discussão

O desempenho dos algoritmos variou drasticamente dependendo do tamanho do vetor e do estado inicial de ordenação dos dados, confirmando as previsões da análise assintótica de complexidade.

3.1. Cenário Aleatório

No cenário puramente aleatório, o **Quick Sort** demonstrou superioridade absoluta à medida que o volume de dados cresceu. Para $N = 10.000$, o Quick Sort completou a tarefa em apenas **0,772 ms**, enquanto o Insertion Sort levou **18,231 ms** e o Bubble Sort necessitou de **61,486 ms**.

- **Por quê?** O Quick Sort opera com complexidade média de $O(N \log N)$, dividindo eficientemente o problema em subproblemas menores. O Bubble Sort e o Insertion Sort operam em ordem quadrática $O(N^2)$ neste cenário, realizando um número massivo de comparações e trocas/deslocamentos.

3.2. Cenário Crescente

No cenário onde os dados já estavam ordenados de forma crescente, ocorreu uma inversão marcante de desempenho: o **Insertion Sort** foi o algoritmo mais eficiente de todos, finalizando o vetor de 10.000 elementos em impressionantes **0,056 ms**. Em contrapartida, o **Quick Sort** apresentou seu pior desempenho geral, demorando **26,725 ms**.

- **Por quê?** O Insertion Sort possui complexidade de melhor caso de $O(N)$. Quando o array já está ordenado, o laço interno de verificação nunca é satisfeito, exigindo apenas uma passagem simples pelo vetor sem deslocar nenhum elemento.
- O Quick Sort decaiu para o seu pior caso de complexidade ($O(N^2)$). Como a regra imposta determina a escolha do **último elemento como pivô**, selecionar o maior elemento de um vetor ordenado gera partições severamente desbalanceadas (uma partição vazia e outra com $N - 1$ elementos). Isso faz com que a árvore de recursão atinja uma profundidade linear de quase 10.000 níveis, degradando o tempo e exigindo alta alocação de memória de pilha.

3.3. Cenário Decrescente

No cenário inversamente ordenado, o **Insertion Sort** superou novamente os demais nos vetores maiores (**9,292 ms** para $N = 10.000$). O Bubble Sort obteve o pior tempo registrado em todo o experimento, atingindo **92,649 ms**.

- **Por quê?** O Bubble Sort sofre neste cenário porque precisa realizar a operação de troca (**swap**) em absolutamente todas as comparações possíveis, estourando o limite de operações de escrita na memória. O Quick Sort sofre do mesmo problema do caso crescente: ao pegar o último elemento (que é o menor valor do vetor restante), as partições ficam totalmente desbalanceadas, gerando novamente uma degradação para $O(N^2)$.

4. Conclusão

Os testes práticos ratificaram os conceitos teóricos de estruturas de dados:

1. O **Quick Sort** é ideal para grandes volumes de dados arbitrários ou aleatórios, mas sua estratégia clássica de pivô fixo no final o torna altamente vulnerável a dados previamente ordenados.

2. O **Insertion Sort** apresenta uma eficiência mecânica excelente para vetores quase ou totalmente ordenados, superando algoritmos avançados nessas condições específicas.
3. O **Bubble Sort** confirmou ser o algoritmo menos eficiente para propósitos gerais, escalando de forma custosa em cenários de pior caso ($O(N^2)$) devido ao excesso de trocas físicas na memória.